Measuring the Cost of the Spectral Relaxation in Normalized-Cut Image Segmentation

Bekay Oryngazy

September 2026

Abstract

Spectral clustering turns image segmentation into an eigenvalue problem, and that the relaxation can be loose has been known since Guattery and Miller [10]. What has not been reported is the size of the effect on a graph a practitioner would build, or which part of the weight function is responsible. This study asks one question of the Shi–Malik image graph. The question is how much the relaxation costs, in the units of the objective itself, and which part of the weight function sets that cost.

The answer is that the eigenvectors handed to k-means are the cheapest functions on the graph and not the cheapest cuts, and the two orderings differ because Dirichlet energy is quadratic. Spreading one unit of change across m edges costs $1/m$ of concentrating it on a single edge, so a gradual ramp across a large lattice can undercut the indicator of a genuinely cheap object boundary. Measured on a standard test image, the solver returns a partition scoring $\mathrm{Ncut} = 2.36 \times 10^{-2}$ against 3.91×10^{-3} for the cheapest single-object partition it passed over. That gap is a factor of 6.0 in the objective the method is written to minimize. At a tighter bandwidth five of the first ten eigenvectors each carry between 92 and 98 per cent of their energy on one single object. The bandwidth is therefore the parameter that sets the price of a boundary and decides which family of function reaches the bottom of the ordering first. The measurement repeats on a second scene with a five dimensional feature space, where the gap is 3.2 to 3.9. The same reading is offered for both walls of a usable bandwidth window less than one decade wide, although the two energies behind that reading were not compared directly. It does not account for halving the image resolution, which runs 7.3 times faster and raises ARI from 0.345 to 0.960. Four predictions were registered before the measurements and a fifth after the mechanism was understood. Four of the five failed at least in part, and two earlier conclusions of this project are reversed.

Keywords: spectral clustering, normalized cuts, image segmentation, graph Laplacian, eigenvector localization, robot perception.

1. Introduction

1.1. Motivation and Problem Origin

This project started with a robot rather than with mathematics.

FIRST Tech Challenge is a robotics competition in which student teams build a robot to play a game that changes every year. The 2025–26 game scores balls of two colors far more highly when they are collected in an order revealed at the start of the match, so the autonomous routine has to collect them in a particular order rather than quickly.

We wrote the vision code for that routine. The camera segmented each frame by color, and every connected region of one color was treated as one artifact, which worked until two artifacts of the same color came to rest touching each other. From then on, they arrived at the planner as a single region, and if the ball the motif called for was sitting inside a cluster of its own color the routine could not pick it out at all. We conceded the problem in the mechanism, declaring same-colored neighbors within ten to fifteen centimeters one object and sorting the cluster with a color sensor further down the intake.

That fix worked, and it is also the reason this study exists, since the discrimination had moved to a sensor downstream of the camera. No threshold placed on color can ever put a boundary between two objects that

have the same color. The failure was therefore not a bug in the implementation, and not a parameter that had been set badly. The pipeline was answering a different question from the one the robot needed an answer to.

1.2. Limitations of Centroid-Based Clustering

The vision code was, in effect, running k-means on color. Given points $x_1, \ldots, x_n$ and a number of clusters k, k-means chooses centers $\mu_1, \ldots, \mu_k$ and an assignment c that minimize the within-cluster sum of squared distances

$$J(c, \mu) = \sum_{i=1}^{n} \left\| x_i - \mu_{c(i)} \right\|_2^2. \tag{1}$$

A point goes to whichever center is nearest, so the region claimed by center j is the intersection of the half-spaces that place j closer than every other center, and an intersection of half-spaces is convex. Every cluster k-means can produce is therefore convex, and no choice of k, no initialization and no tuning changes that.

Two artifacts of the same color that touch each other occupy one convex blob in color space and share a centroid exactly, and two concentric rings have the same center. In both cases the structure humans see is a property of which points are joined to which and not of where the points sit. Since k-means has no representation of joining, the objective it minimizes is a statement about proximity, and no minimizer of that objective can express joining. The question the robot needed answered was about connectivity.

1.3. Spectral Clustering as an Alternative Formulation

Spectral clustering does not answer the proximity question more accurately. It replaces the question with a different one. Build a graph whose vertices are the pixels and whose edge weights are large between pixels that are similar and close, then look for the places where the graph can be cut without spending much weight. Two touching balls of the same color are joined only by a thin bridge of contact pixels, so the cut there is cheap even though the two sides are identical in color. The graph sees connectivity, which is the one thing that separates them.

Finding the cheapest balanced cut exactly is NP-hard, and the standard remedy is to relax the problem into an eigenvalue problem [23]. That relaxation dates from 2000. What it buys is a new object to reason about. The partition is no longer chosen by a rule that a practitioner writes down. It is read out of the bottom of a spectrum, and the practitioner's only remaining influence is the weight function that decided what the graph was in the first place.

1.4. The Research Question

The literature settles the mathematics of that step, and it settles a good deal about how to choose its parameters [20, 30, 31], and Guattery and Miller [10] settle that the relaxation can be badly loose. What no source found here reports is how loose it is on a graph built from an image at a bandwidth someone would choose. A practitioner sets a bandwidth and receives a partition, with no instrument reporting which of the features in the weight produced it.

This study asks one question, which is what the relaxation costs on such a graph in the units of the objective the method is written to minimize, and which part of the weight function sets that cost. The answer turns out to be one mechanism, derived in Section 4.4 and measured in Section 6.3.2, that shows up in experiments designed for different reasons. Whether the graph earns its runtime is answered as a consequence, since once the mechanism is clear it also explains which of the available speedups work and which do not.

2. Definitions and Notation

Everything in this study is built from nine objects. They are collected here so that the argument can be read without stopping, and each one is stated in the plainest form that is still correct.

Definition 1 (Weighted graph and degree)**.** A weighted graph is a set of vertices together with a symmetric matrix W whose entry $w_{ij} \geq 0$ says how strongly vertex i and vertex j are joined. The degree $d_i = \sum_j w_{ij}$ is

the total weight attached to i, and D is the diagonal matrix holding those degrees. In this study a vertex is one pixel.

Definition 2 (Cut and association)**.** For a set A of vertices and its complement $\bar{A}$,

$$\operatorname{cut}(A,\bar{A}) = \sum_{i\in A,\, j\in\bar{A}} w_{ij}, \qquad \operatorname{assoc}(A) = \sum_{i\in A}\sum_{j} w_{ij}.$$

The cut is the weight that has to be paid to separate A from the rest. The association is the total weight touching A, which measures how much graph A holds.

Definition 3 (Normalized cut)**.**

$$\operatorname{Ncut}(A,\bar{A}) = \frac{\operatorname{cut}(A,\bar{A})}{\operatorname{assoc}(A)} + \frac{\operatorname{cut}(A,\bar{A})}{\operatorname{assoc}(\bar{A})}. \tag{2}$$

Dividing by the association is what stops the objective from shaving off single vertices, since a shaved vertex has a small numerator and an equally small denominator.

Definition 4 (The three Laplacians)**.** $L = D - W$ is the unnormalized Laplacian, $L_{\text{sym}} = D^{-1/2} L D^{-1/2}$ is the symmetric normalization, and $L_{\text{rw}} = D^{-1} L$ is the random-walk normalization. L and L_{sym} are symmetric positive semidefinite. L_{rw} is similar to L_{sym}, so its eigenvalues are the same real non-negative numbers even though the matrix itself is not symmetric. For all three, the multiplicity of the eigenvalue zero equals the number of connected components.

Definition 5 (Dirichlet energy)**.** For any $y \in \mathbb{R}^n$,

$$y^{\top} L y = \tfrac{1}{2} \sum_{i,j} w_{ij} (y_i - y_j)^2. \tag{3}$$

This is the total disagreement of y across the graph, weighted by similarity. It is never negative, and it is zero exactly when y is constant on every connected component.

Definition 6 (Rayleigh quotient)**.**

$$R(y) = \frac{y^{\top} L y}{y^{\top} D y}, \tag{4}$$

which is disagreement divided by how much graph the function occupies. This study reads it as a price per unit of connection, and the whole argument turns on that reading.

Definition 7 (Spectral embedding and eigengap)**.** The spectral embedding sends vertex i to the row $(v_1(i), \ldots, v_k(i))$ built from the k smallest eigenvectors, and k-means is run on those rows. The eigengap is $\lambda_{k+1} - \lambda_k$, and it bounds how far the bottom k eigenvectors can rotate under a perturbation of the matrix.

Definition 8 (The image weight function)**.** Following Shi and Malik [23], two pixels within a radius r of each other are joined with weight

$$w_{ij} = \exp\!\left(-\frac{\|F_i - F_j\|_2^2}{2\sigma_I^2}\right) \cdot \exp\!\left(-\frac{\|X_i - X_j\|_2^2}{2\sigma_X^2}\right), \tag{5}$$

and pixels further apart than r are not joined at all. F_i is the feature vector of pixel i and X_i is its position. The two bandwidths σ_I and σ_X set how quickly similarity falls off in appearance and in space.

Definition 9 (Boundary weight fraction and participation ratio)**.** ρ is the fraction of all edge weight in the graph that crosses the true boundary of the ground-truth partition, so it reports what the weight function did to the boundary rather than what the algorithm did with it. The participation ratio $\operatorname{PR}(v) = n^{-1} (\sum_i v_i^2)^2 / \sum_i v_i^4$ is the fraction of vertices an eigenvector occupies, and a value near 1 means the eigenvector is spread over everything while a value near 0 means it sits on a small set.

Partitions are scored with the adjusted Rand index [12] and with adjusted mutual information [28], both of which subtract the agreement expected by chance, so that zero means chance and one means exact. Boundary quality is scored separately with a boundary F-measure at a two-pixel tolerance.

Table 1. Notation used throughout.

Symbol	Meaning
W, D, L	weights, degree matrix, unnormalized Laplacian $D - W$
L_{sym}, L_{rw}	$D^{-1/2}LD^{-1/2}$ and $D^{-1}L$
σ_I, σ_X, r	feature bandwidth, spatial bandwidth, radius cutoff
F_i, X_i, I	feature vector, position of pixel i, CIELab lightness
λ_j, v_j	jth smallest eigenvalue and its eigenvector
$R(y)$	Rayleigh quotient $y^\top Ly / y^\top Dy$
ρ	fraction of edge weight crossing the true boundary
$\text{PR}(v)$	participation ratio, the fraction of vertices v occupies
ARI, AMI	chance-corrected agreement with ground truth

3. Literature Review and Background

3.1. Overview

Recent and foundational work related to normalized-cut image segmentation can be grouped into five directions. First, the objective itself and its relaxation to an eigenvalue problem establish what the method computes. Second, a group of studies establishes why the bottom of the spectrum is the right place to read a partition from, and what the eigengap contributes to that reading. Third, a separate literature quantifies how far the relaxed solution can fall from the discrete optimum, which is the direction this study measures. Fourth, work on eigenvector localization asks when an eigenvector that sits on a small set is tracking real structure, which is the behavior Section 6.4 reports. Finally, work on bandwidth selection, sampling, and recent extensions of the objective describes how the method is used in practice.

3.2. The Normalized Cut Objective and Its Relaxation

Minimizing the cut of Definition 2 on its own is degenerate, because the cheapest cut in almost any graph shaves off a single poorly connected vertex. Shi and Malik [23] removed the degeneracy by dividing, which is Definition 3. Minimizing that objective over all partitions is NP-hard, and relaxing the indicator vector to take real values gives the generalized eigenproblem $(D - W)y = \lambda Dy$, whose second-smallest solution is the object every implementation computes. The term that removes one pathology is also the term that introduces the looseness this study measures.

3.3. Foundations of Spectral Clustering and the Role of the Eigengap

Four papers arrive at the same computation from four different directions, and each of them contributes a different reason to trust it.

Ng et al. [20] start from the case in which the answer is already known. If a graph has k pieces with no edges between them, its affinity matrix is block diagonal, the eigenvalue zero has multiplicity k, and the corresponding eigenvectors are indicators of the pieces. Weak rather than absent connections between the pieces make the matrix a perturbation of that one, so the eigenvectors are a perturbation of the indicators, and the size of the perturbation is controlled by the gap between the kth and the $(k+1)$st eigenvalue, which is the Davis–Kahan theorem [6]. That gap decides whether the argument applies at all, so this study reports it wherever it uses one.

Belkin and Niyogi [3] arrive from the opposite side. They ask which functions on a graph vary least across edges, and show that the answer is the bottom eigenvectors of the Laplacian. The consequence is easy to miss. If the weight function assigns most of its weight according to spatial adjacency, the smoothest functions on that graph are smooth functions of position, and the resulting partition will be a partition of the pixel grid whether or not the image contains any objects.

Coifman and Lafon [4] put a random walk on the graph and define a distance between vertices in terms of how likely the walk is to connect them. The eigenvalues become time scales and a cluster becomes a

region the walk is slow to leave, which is what makes the bandwidth intelligible, since raising it lets the walk run longer and a walk that runs long enough stops distinguishing anything. von Luxburg [30] compares the three Laplacians used in practice and recommends the random-walk normalization on the grounds that the unnormalized version is sensitive to variation in degree, and this study measures that recommendation and does not reproduce its ordering.

3.4. Known Bounds on Relaxation Quality

The relaxation is not exact, and the size of the error has a literature of its own. Guattery and Miller [10] construct a graph, since known as the roach, which is a ladder with half of its rungs removed so that it has a long body and two long antennae. The optimal bisection severs the antennae and cuts two edges, while the median cut of the Fiedler vector runs the length of the body and cuts $n/4$ edges, because the cheapest function on that graph is a ramp along the long axis rather than the indicator of the cheap boundary. That is the mechanism Section 4.4 derives, on a graph built to display it, and the Cheeger inequality [1] brackets the same error from both sides. The roach is a construction and the bound is worst-case, so what is missing is the size of the effect on a graph a practitioner would actually build, in units of the objective itself.

3.5. Eigenvector Localization

One of the results below is an eigenvector that sits almost entirely on one object. That behavior has its own literature. Cucuringu and Mahoney [5] study localization in eigenvectors away from the extreme ends of the spectrum and find that it sometimes tracks interpretable structure and sometimes does not, which is why this study reports localization as an observation and not as a detector. Nadakuditi and Newman [19] show, for community detection, that below a certain point the informative eigenvector stops being distinguishable from the bulk, so where a structure sits in the ordering decides whether an algorithm can see it at all. Krzakala et al. [14] give the case where localization is the problem rather than the signal, since on sparse graphs with uneven degrees the leading eigenvectors concentrate on a few high-degree vertices, and Saade et al. [22] answer that with the Bethe Hessian. This study measures position in the ordering directly, which is the quantity all four of these studies turn on when they ask whether a localized eigenvector carries information.

3.6. Bandwidth Selection and Local Scaling

The bandwidth is the parameter that decides how similar two pixels have to be before the graph joins them, and Zelnik-Manor and Perona [31] treat the choice of a single global value for it as the weak point of the method. They answer that with local scaling. Instead of one global bandwidth, each point s_i receives its own $\sigma_i = d(s_i, s_K)$, the distance to its Kth neighbor with $K = 7$, and the weight becomes

$$w_{ij} = \exp\left(-\frac{d(s_i, s_j)^2}{\sigma_i \sigma_j} \right). \tag{6}$$

The idea works because it removes the assumption that one scale fits the whole data set, which is the assumption a global bandwidth quietly makes. The same paper also proposes choosing the number of clusters by rotating the eigenvectors to minimize an alignment cost, which is the direct competitor to the eigengap test used here and is not tested. Local scaling is used as one of four bandwidth rules.

3.7. Sampling Methods and Recent Extensions of the Objective

The cost of the eigenproblem has been attacked mainly by sampling. Fowlkes et al. [8] extend a partition computed on a subset to the rest of the data, and a later survey gives conditions under which that is safe [26]. Shi and Malik [23] make a related claim about sparsification, which Section 6.8 examines directly because the version of it that circulates is not the version they wrote.

Tang et al. [25] use the normalized cut as a differentiable loss for weakly supervised segmentation, Melas-Kyriazi et al. [18] build the same graph on self-supervised transformer features instead of color, and recent work continues in both directions [11]. What changes in those papers is the feature space that feeds the weight

function, while the Laplacian, the eigensolve and the objective are unchanged, so the mechanism described here is not a statement about an old method. It should apply wherever a normalized cut is taken on a graph whose weights encode similarity, including graphs whose features come from a network. That is an argument rather than an experiment, because the pretrained weights were not reachable from the environment this work ran in, and Section 6.3.4 does the next best thing by repeating the central measurement in a five dimensional feature space.

What the sources above establish is that the relaxation is loose and by how much in the worst case. What none of them reports is the size of the effect on an image graph at a bandwidth someone would choose, or which of the features in the weight function is responsible for it. A practitioner who sets a bandwidth, runs the pipeline and gets a plausible-looking partition has no diagnostic for either, and that is the gap this study measures with the instruments of Section 5.

4. Theoretical Framework

This section establishes what an eigenvector of a graph Laplacian represents and why the bottom of the spectrum is an ordering of functions. It contains no measurements, and it exists because the results in Section 6 are not intelligible without it. The single fact that explains all of them is a fact about what an eigenvector of a Laplacian is for.

4.1. The Laplacian as a Measure of Disagreement

Let W hold the edge weights and let D be the diagonal matrix of degrees $d_i = \sum_j w_{ij}$. The unnormalized Laplacian is $L = D - W$, and its action on a function y defined on the vertices is

$$(Ly)_i \;=\; d_i y_i - \sum_j w_{ij} y_j \;=\; \sum_j w_{ij}\,(y_i - y_j). \tag{7}$$

At vertex i, the Laplacian reports how far y_i sits from the values of its neighbors, with each neighbor counted according to how strongly it is joined. A function that agrees with its neighbors everywhere is in the null space. The random-walk normalization makes the reading even plainer, since $L_{\mathrm{rw}} = D^{-1}L = I - D^{-1}W$ and $D^{-1}W$ is the operator that replaces y_i by the weighted average of its neighbors. Therefore $L_{\mathrm{rw}}y$ is the difference between a function and its own local average, which is the discrete version of $-\nabla^2$.

4.2. Eigenvalues as Normalized Energy Costs

Suppose $Ly = \lambda Dy$, which is the generalized problem the relaxation actually produces. Multiplying on the left by $y^\top$ and dividing gives $\lambda = R(y)$, the Dirichlet energy of y divided by how much graph y occupies. In the unnormalized case D is the identity and the eigenvalue is the Dirichlet energy itself. Either way an eigenvalue is the price, in weight, of the pattern of disagreement that its own eigenvector describes, and every eigenvalue reported in Section 6 is of the generalized kind. Small eigenvalues belong to functions that change slowly across edges, and large eigenvalues belong to functions that alternate rapidly, which is why the spectrum is often read as a frequency ladder.

If y is the indicator of a set A, the same formula gives $y^\top Ly = \mathrm{cut}(A, \bar{A})$. A cut is therefore a special case of Dirichlet energy, namely the case where all of the disagreement happens across one boundary.

4.3. The Rayleigh Quotient and the Variational Ordering

Eigenvectors are usually introduced as solutions of an equation, and the reading this study uses is the variational one. The Rayleigh quotient of Definition 6 is disagreement divided by how much graph the function occupies, which is a price per unit of connection. The Courant–Fischer theorem [24] says that the smallest generalized eigenvalue is the smallest value of R, the second smallest is the smallest value of R among functions D-orthogonal to the first, and so on up the ladder.

That is a procedure and not a formula. The first eigenvector is the cheapest way for a function to disagree with itself, the second is the cheapest way that is independent of the first, and each one after that repeats the

step. The bottom of the spectrum is therefore a list of candidate ways to divide the graph, sorted from cheapest to most expensive. Spectral clustering takes the first k entries of that list and hands them to k-means, which is the whole of the algorithm. Everything this study reports is a statement about what ends up near the top of the list and why.

4.4. Quadratic Energy and the Advantage of Distributed Change

The list is not a list of cuts. It is a list of functions, and that difference is the mechanism behind every experiment in Section 6.

Take a path of m edges, each of weight w, and ask a function to rise from 0 at one end to 1 at the other. If it rises in a single step, one edge carries the whole change and the Dirichlet energy is $w \cdot 1^2 = w$. If it rises in m equal steps, each edge carries $1/m$ of the change and the energy is

$$m \cdot w \cdot \left(\tfrac{1}{m}\right)^2 \;=\; \frac{w}{m}. \tag{8}$$

The two functions cover the same range, and their denominators in Equation (4) differ only by a small constant factor, so the ramp is cheaper by approximately a factor of m, because Equation (3) is quadratic. Splitting a change into many small pieces is rewarded, while concentrating it into one large piece is punished, because squaring a small per-edge difference shrinks it faster than adding more edges grows the sum.

An indicator vector is not allowed to spread, since it is 0 on one side and 1 on the other and all of its change is therefore concentrated on the boundary. A relaxed eigenvector has no such constraint. Therefore the relaxation is not a slightly blurred version of the discrete problem. It is a different problem, whose cheapest solutions are gradual functions that no partition corresponds to, and the two orderings can disagree.

This is the roach graph of Guattery and Miller [10] stated as arithmetic rather than as a construction, and the two orderings disagree most in one particular case, which is the case this study is about. A region that is small and whose boundary is nearly weightless is *cheap as a cut* and *expensive as a smooth function*, because it forces its change into a short boundary rather than distributing it across the picture, while a gentle ramp across the whole image is the opposite. A small object can therefore be the best partition available and at the same time absent from the bottom of the spectrum. Section 6.3.2 measures that gap on a photograph.

4.5. The Role of Normalization

Minimizing the cut alone is degenerate, and Equation (2) fixes it by dividing. The useful way to see the division is as a change of units. $\mathrm{assoc}(A)$ is the volume of A measured in weight rather than in vertices, so normalizing measures the size of a piece by how much connection it holds. The corresponding change in the eigenproblem is the change from $\|y\|_2^2 = \sum_i y_i^2$ in the denominator to $y^\top D y = \sum_i d_i y_i^2$, which charges a function more for putting its mass on well connected vertices.

An earlier version of this project's own analysis attributed the result of Section 6.3 to this division, and that explanation is wrong. Normalization does penalize a small set, since a small set has a small association and the ratio cut/assoc has a small denominator. However, the numerator of a nearly weightless boundary is smaller still, and Section 6.3 measures both and finds the coin partitions scoring better than the balanced background partition. The bias lives in the relaxation rather than in the objective, for the reason given in Section 4.4.

4.6. Bandwidth as the Price of a Boundary

The weight function used throughout is Definition 8, a product of a feature term and a spatial term with a hard radius cutoff.

The bandwidth σ_I is an exchange rate, since it converts a difference in appearance into a loss of similarity and sets the feature distance at which two pixels stop counting as neighbors. Following Coifman and Lafon [4], σ_I^2 can also be read as a diffusion time, so raising it lets a random walk run longer before the graph is inspected, and a walk that runs long enough stops distinguishing anything.

Section 4.4 says what that means for the ordering. The bandwidth sets the price of a boundary, and it therefore decides whether any region indicator is cheap enough to reach the bottom of the list before the ramps do. That single sentence is the reading offered for both walls of the usable window, and it is the reason the same image can be clustered into objects at one bandwidth and into a grid at another.

The eigengap of Definition 7 is what says whether that ordering can be trusted, since a perturbation of the matrix rotates the bottom eigenvector subspace by an amount bounded above by the size of the perturbation divided by the gap [6, 30]. A large gap therefore reports that the bottom k eigenvectors are a stable description of the graph, which is a statement about stability and not about correctness.

4.7. Registered Hypotheses

The account above was assembled from measurements, and an account assembled that way can explain anything after the fact. To keep the difference between an explanation and a confirmation visible, this study registered four hypotheses in advance, and a fifth after the mechanism of Section 4.4 was understood, and it reports each verdict where the test happens. **H5** is flagged below, because a prediction written after the account it tests is weaker evidence than one written before. A hypothesis with any refuted clause is counted below as failed. Every hypothesis is picked up again in a boxed note at the place in Section 6 where the evidence arrives.

H1. *The eigengap can be used to choose the bandwidth, and not only to describe it after the fact.* Tested in Section 6.5, where it is refuted.

H2. *Bandwidth failure is a threshold rather than a slope, and the threshold sharpens as the number of points grows.* Tested in Section 6.5, where the threshold clause is confirmed while the sharpening clause is refuted.

H3. *The runtime exponent follows the separator theorem [16], giving $n^{1.5}$ on a two-dimensional lattice and n^2 in three dimensions.* Tested in Section 6.8, where the three-dimensional half holds while the two-dimensional half fails.

H4. *Recursive bisection recovers narrow bottlenecks that simultaneous k-way clustering misses, as Shi and Malik [23] claim.* Tested in Section 6.6, where it is confirmed.

H5. *Coarsening the image shortens the lattice, so fewer ramps fit below any given energy and the object-shaped eigenvectors should move down the ordering.* This one was written after the mechanism of Section 4.4 was understood, and it is the only prediction that account has so far generated. Tested twice in Section 6.8 and refuted both times, after which Section 7 treats the result it was meant to explain as this study's largest open question.

Four of the five hypotheses failed at least in part, and only **H4** survived intact. That ratio is reported rather than buried, since a registered prediction that always succeeds is usually one that was written afterwards.

5. Method

5.1. Overview

The method has four components. A graph is built over the input, using the Shi–Malik weight of Definition 8 wherever that input is an image, while three experiments use other graphs and say so where they appear, namely a k-nearest-neighbor graph on point sets in Sections 6.2 and 6.5 and an unweighted occupancy grid in Section 6.9. Three diagnostic instruments are then applied, each built because a specific question could not be answered from output partitions alone. Four label-free bandwidth rules were implemented and three Laplacians are compared, and every result below uses the median rule unless a row says otherwise, with every eigenpair coming from ARPACK in shift-invert mode. Partitions are scored against ground truth with chance-corrected agreement measures and a boundary F-measure.

5.2. Graph Construction

Every pixel becomes a vertex, and the feature vector F_i is the CIELab color triple, which is used because Euclidean distance in that space corresponds roughly to perceived difference, so one bandwidth means some-

thing similar in dark and bright regions. Where texture is used, F_i is extended by two texture-energy channels computed as $\sqrt{G_{2s} * (I - G_s * I)^2}$, where I is the CIELab lightness channel, G_s is a Gaussian kernel of standard deviation s pixels, $*$ is convolution and $s \in \{1, 2.5\}$ pixels. The position vector X_i is the pixel coordinate, and the weights follow Definition 8. The radius r is what makes the matrix sparse, since it decides which entries are nonzero before either Gaussian is evaluated. The coins image is the *coins* sample of scikit-image [27] rescaled by 0.35, which gives $n = 14{,}204$ pixels, and it is used throughout with $r = 5$ pixels, $\sigma_X = 4$ and a mean degree of 77.1.

5.3. Diagnostic Instruments

The question in Section 1.4 cannot be answered by looking at output partitions, so this study builds three small instruments and uses them throughout.

The first is a switch. Since Equation (5) is a product, sending $\sigma_I \to \infty$ sets the feature factor to one and leaves a purely spatial graph, and sending $\sigma_X \to \infty$ leaves a purely feature-based graph. The sparsity pattern is identical in all three cases, since r and not either Gaussian decides which entries exist. Therefore the three graphs are directly comparable, and any difference between their partitions is attributable to the factor that was switched off.

The second is a measured quantity rather than a knob, namely the fraction of total edge weight that crosses the true boundary of the ground-truth partition. It reports what the weight function did to the boundary, and it does so on the graph rather than on the output.

The third is a threshold, since floating-point weights below 10^{-12} are treated as absent when connected components are counted. An early version of this project counted a graph as connected when its weakest edges were of order 10^{-78}, which is connected in the sense that the sparsity pattern is connected and disconnected in every sense that matters to an eigensolver.

5.4. Bandwidth Rules, Algorithms, and Solvers

To avoid choosing σ_I by looking at the answer, four label-free rules are used: the median pairwise feature distance in a sample, the distance to the seventh nearest neighbor, the fifteenth percentile of the pairwise distance range, and the local scaling of Equation (6). The median rule is the default and is the one reported unless stated otherwise.

The pipeline itself is the standard one. Build W, form the chosen Laplacian, compute the k smallest eigenpairs, row-normalize the embedding where L_{sym} is used, and run k-means in that embedding. Eigenpairs come from ARPACK in shift-invert mode [15], with the shift set to -10^{-8} times the largest absolute diagonal entry of the Laplacian, so that the factorization does not fail on a graph that has become disconnected. The recursive normalized cutter is implemented directly instead of being taken from a library, because the breadth-first policy Shi and Malik [23] recommend is not what standard implementations do.

5.5. Evaluation Protocol

Partitions are scored with the measures of Section 2. How many times each experiment was repeated matters as much as the scores themselves, so it is stated plainly here rather than in a footnote. The feature ablation is an average over eight images built from different random seeds. The resolution sweep is an average over five, and the edge-dropping comparison over three. Each of those is reported as a mean together with a 95 per cent Student-t interval, which is the interval that is correct at these small sample sizes. Where two methods are compared across the same set of images the comparison is paired, and it is tested with a Wilcoxon signed-rank test over the pairs instead of by reading two intervals against each other, since overlapping intervals do not by themselves establish that a paired difference is unresolved. Everything computed on the coins image is different, and it is worth being precise about which parts of it are deterministic. The graph, the spectrum and every Ncut value evaluated on a named partition are functions of the image and the parameters alone, so they are reported from one run. The partitions that k-means returns from the embedding are not, since k-means is randomly initialized, so every run of it in this study uses ten restarts and a fixed seed of 42, and those

two settings are what make the numbers in Section 6.3 repeatable rather than any claim that the step has no randomness in it.

Everything runs in Python 3.11 on SciPy [29] and scikit-image [27]. The graph construction, the three Laplacians, the solver wrappers, the recursive cutter and every diagnostic were written for this project directly on SciPy sparse matrices on top of NumPy, while scikit-learn [21] supplies k-means and the scores and nothing else, and every figure is drawn with Matplotlib.

6. Experiments and Results

6.1. Overview

Every method reviewed in Section 3 ends at the same fixed sequence of steps, and Figure 1 draws that sequence once. Every failure reported here belongs to one stage of that sequence, and all but one of them has a mechanism attached. The figure also shows which stage that mechanism lives in, which results are consequences of it, and the one result that is not.

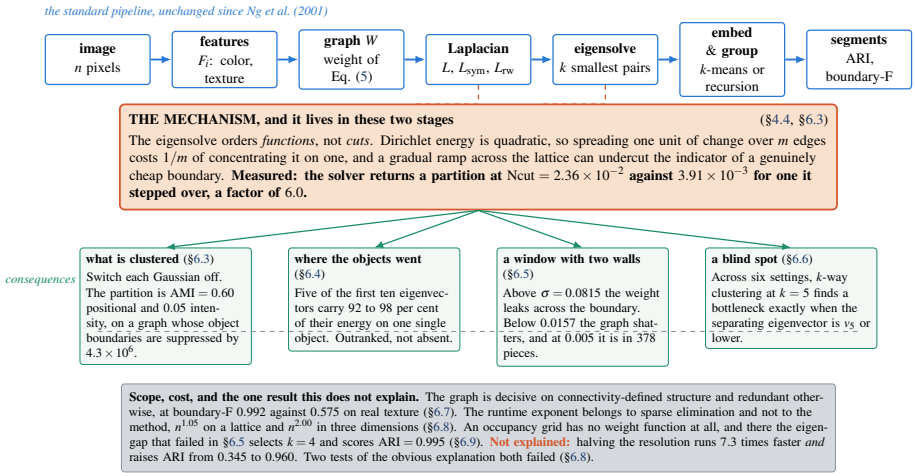


Figure 1. The pipeline this study examines, and where each measurement attaches to it. The top row is the standard computation. The wide band below it is the mechanism and the two stages it lives in. The row of four boxes is what follows from the mechanism. The bottom band is scope, cost, and the one result the mechanism does not account for.

6.2. Baseline Comparison on Synthetic Structure

The first question is whether the extra machinery is ever decisive. On three synthetic sets generated by scikit-learn's *make_blobs*, *make_circles* and *make_moons* at $n = 600$ and noise 0.05, using L_{sym} on a k-nearest-neighbor graph with the median rule, k-means and spectral clustering score 1.000 and 1.000 on three convex blobs, -0.002 and 1.000 on two concentric circles, and 0.239 and 1.000 on two moons.

On the convex blobs both methods score 1.000, and the graph costs two orders of magnitude more time to reach the same answer. That settles the cost question for the convex case. Where the structure is defined by connectivity, k-means scores -0.002 on the circles and 0.239 on the moons, while spectral clustering scores 1.000 on both. These figures indicate that the graph is decisive on connectivity-defined structure and redundant on convex structure.

6.3. Anatomy of the Image Graph

The instrument of Section 5 makes it possible to ask what a partition is a partition *of*. On the standard coins image at $n = 14{,}204$, $k = 4$ and L_{rw}, two reference labelings are constructed: an intensity reference from thresholding, and a positional reference from a k-way split of the pixel coordinates. Table 2 scores the output of three graphs against both.

Table 2. Anatomy of the image graph, $n = 14{,}204$, $k = 4$, L_{rw}. All three graphs have the same sparsity pattern, because the radius cutoff and not either Gaussian decides which entries are nonzero.

Graph	AMI with intensity	AMI with position	Reading
intensity only ($\sigma_X = \infty$)	0.049	0.604	mostly positional
spatial only ($\sigma_I = \infty$)	0.072	**0.985**	purely positional
product ($\sigma_I = 0.1$, $\sigma_X = 4$)	0.049	**0.604**	mostly positional

The product graph, which is the one a practitioner would build, produces a partition that agrees with pixel position at AMI $= 0.604$ and with intensity at 0.049. Switching the spatial factor off entirely changes nothing, since the intensity-only row is identical to three decimal places. A reader may expect a graph with no positional term in its weights to score zero against position, and it does not, because the radius cutoff r makes that graph a lattice whatever σ_X is set to. Locality enters through the sparsity pattern and not through the spatial Gaussian, which is why switching the Gaussian off changes nothing. This indicates that the partition is a partition of the pixel grid rather than of the image, and that nothing a practitioner would have looked at revealed it. No error was raised, and the output was visually consistent with a reasonable segmentation. Drawn as an image, the second eigenvector of the spatial-only graph is a cosine mode of the lattice, and the second eigenvector of the product graph is the same mode with the coins visible only as small perturbations.

6.3.1. Evaluation of the Denominator Explanation

The obvious explanation is that $\sigma_I = 0.1$ was simply too large for intensity to matter, and measurement refutes it. At $\sigma_I = 0.1$ the mean intensity weight on interior edges is 0.742, while on the top two per cent of intensity-jump edges it is 1.7×10^{-7}, a suppression by a factor of 4.3×10^{6}, which deepens the discrepancy instead of resolving it. The intensity factor also contributes 3.8 times more to the mean log-weight than the spatial factor does.

The second explanation, and the one an earlier version of this project's own analysis argued for, is that the normalized cut outranks the coin because a coin is one or two per cent of the image, so dividing a small numerator by a very small denominator gives a large number. That explanation is also wrong, and the test that settles it is direct evaluation of the objective rather than argument from the form of the ratio.

6.3.2. Direct Objective Evaluation on Unselected Partitions

Equation (2) can be evaluated on any bipartition, including ones no algorithm proposed. The graph used from here to the end of this subsection is the intensity-only graph at $\sigma_I = 0.10$, which Table 2 shows returns the same partition as the product graph, so that the bandwidth can later be varied as the single free quantity. On that graph the cheapest single-coin bipartition scores Ncut $= 3.91 \times 10^{-3}$, with a cut of 30.78 against an association of 7,950 for a region holding 1.27 per cent of the pixels, one of the larger coins. A balanced vertical bisection of the image scores 2.90×10^{-2}.

> **Observation 1.** The solver returns a partition at Ncut $= 2.36 \times 10^{-2}$ against 3.91×10^{-3} for the cheapest single-coin partition it passed over, a factor of 6.0 and an increase of approximately 500 per cent in the objective the method is written to minimize. The coin cut is outranked by the relaxation rather than by the normalized cut.

Section 4.4 predicts this and says why, and Figure 2 shows it. The second eigenvector has $\lambda_2 = 1.12 \times 10^{-3}$, below the coin partition's 3.91×10^{-3} as any relaxation must be, and a participation ratio of 0.51, which places it on 51 per cent of the vertices. It is a gradual ramp from one side of the image to the other, which is the cheapest thing a function can be on a large lattice, and the third eigenvector is another such ramp. The first eigenvector carrying more than half its energy on a single coin is v_7, at a participation ratio of 0.012, and v_4 falls below that half-energy threshold at 0.49. At $k = 4$ the solver therefore receives ramps and nothing else,

and k-means in that embedding returns a partition of the grid.

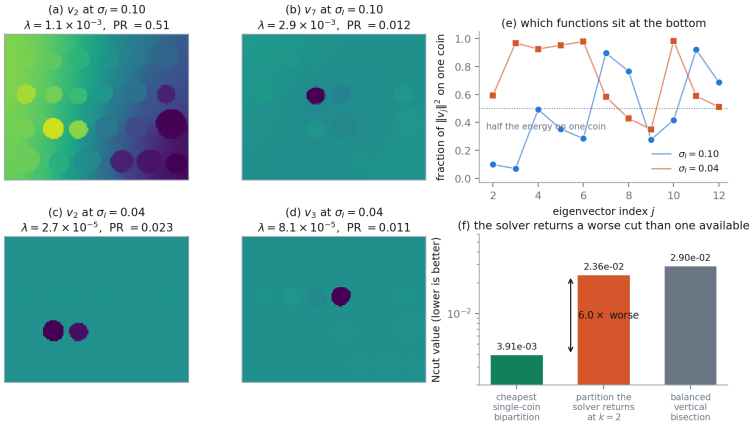


Figure 2. Why the objects are outranked. Panels (a) to (d) show four eigenvectors drawn as images. At $\sigma_I = 0.10$ the cheapest function is a gradual ramp across the whole lattice, and the first object-shaped function is v_7. At $\sigma_I = 0.04$ the boundary is cheap enough that object indicators take the bottom places instead. Panel (e) tracks the reordering. Panel (f) evaluates Equation (2) on three bipartitions of the same graph.

The same figure shows the control that makes the account testable. Lowering the bandwidth to $\sigma_I = 0.04$, with the same image, the same radius, the same objective and the same reference coin, drops that coin's boundary cut from 30.78 to 3.54, and the ordering inverts. Now v_2 carries 0.59 of its energy on one coin, v_3 carries 0.97, and the participation ratios fall to 0.023 and 0.011. One quantity was varied and the bottom of the spectrum changed families, so the price of a boundary is what decides which family reaches the bottom of the list first.

6.3.3. Cluster Count and Spatial Bandwidth

If the object cuts are present and merely outranked, raising k should let them in, because a longer list reaches further down. On a 61×77 crop of the same image at $n = 4{,}697$, raising k from 4 to 12 moves the agreement with the intensity reference from 0.083 to 0.254 and the agreement with the positional reference from 0.764 down to 0.527. Sweeping σ_X across two orders of magnitude moves the positional agreement only from 0.88 to 0.76. Since raising k helps and no spatial bandwidth does, the constraint sits in the ordering rather than in the weight.

The reading is weaker than a clean switch would have been, since sweeping σ_I alone never makes this image cleanly intensity-driven at $k = 4$. The intensity reference peaks at only AMI $= 0.200$ at $\sigma_I \approx 0.023$, where the positional reference already stands at 0.149.

6.3.4. Replication on a Second Scene

One image with one scalar feature is a thin base for a claim about a mechanism, so the whole measurement was repeated on the texture benchmark of Section 6.7, which has five regions instead of twenty-four objects, exact labels, and a five dimensional feature vector of CIELab color together with two texture-energy channels.

One methodological point had to be settled first. At the plain median rule the two weakest edges holding this benchmark's graph together carry 9.3×10^{-9} and 1.7×10^{-8}, so it passes the 10^{-12} test of Section 5 and its second eigenvalue is 2×10^{-12}, which is numerical noise rather than a description of the data. The threshold was therefore raised for this scene, and the bandwidth was raised until the graph had a single component after discarding weights below 10^{-8} and a second eigenvalue above 10^{-5}. That criterion uses no labels, and it selects three times the median in every seed. The same defect, in a milder form, is present in this project's own earlier graphs.

On that graph, over three seeds, the cheapest true-region bipartition scores Ncut between 2.03 and 2.59 $\times$

10^{-3} while the partition the solver returns at $k = 2$ scores between 7.42 and 9.49×10^{-3}. The solver is therefore between 3.2 and 3.9 times more expensive than a partition available to it, against a factor of 6.0 on the coins. In all three seeds λ_2 lies between 2.86×10^{-4} and 3.06×10^{-4} with a participation ratio between 0.53 and 0.55, which is again a ramp across half the picture. The first eigenvector carrying more than half its energy on a single non-background region appears at index four in one seed, index five in another, and nowhere in the first sixteen in the third. The same ordering therefore survives a change of image and a move from one feature dimension to five. It is a replicated observation rather than a replicated intervention, since the one-variable bandwidth control of Section 6.3.2 was run only on the coins.

6.4. Eigenvector Localization on Individual Objects

Section 4.4 says the objects should be in the spectrum and not missing from it, sitting above the smooth modes. Prof. Murphy put the same expectation in a sharper form in a July 2026 session, before any of this was measured. His suggestion was to look at the kth eigenvector instead of the second, because a higher-frequency eigenvector can be hot on one circle and cold everywhere else, and that can only happen through the intensities.

The first 24 eigenvectors of L_{rw} were computed on three graphs and scored by participation ratio and by how much of $\|v\|^2$ falls on a single coin, and Table 3 reports the result while Figure 3 draws it.

Table 3. Localization of the low eigenvectors, $n = 14{,}204$, L_{rw}. Participation ratio is the fraction of vertices the eigenvector occupies. The last column is the fraction of $\|v\|^2$ lying on the single coin that best matches that eigenvector.

Graph	Eigenvector	PR	Energy on one coin
intensity only, $\sigma_I = 0.04$	v_3	0.011	**0.97**
intensity only, $\sigma_I = 0.04$	v_4	0.010	**0.92**
intensity only, $\sigma_I = 0.04$	v_5	0.010	**0.95**
intensity only, $\sigma_I = 0.04$	v_6	0.012	**0.98**
intensity only, $\sigma_I = 0.04$	v_{10}	0.017	**0.98**
intensity only, $\sigma_I = 0.10$	v_7	0.012	0.90
intensity only, $\sigma_I = 0.10$	v_{11}	0.011	0.92
spatial only	v_3, the best of the first 24	0.671	0.16

Five of the first ten eigenvectors of the tightly tuned intensity graph carry between 92 and 98 per cent of their energy on one single coin, at participation ratios between 0.010 and 0.017. A participation ratio of 0.011 on 14,204 pixels corresponds to 156 pixels, which is the size of one coin. The spatial-only graph never exceeds 0.16 on any coin, and the eigenvector that reaches that value has a participation ratio of 0.671. This indicates that localization is available through the feature term and not through the spatial term, exactly as predicted. The eigenvectors of a plain lattice are cosine modes, and a cosine mode is spread out by construction.

This result is an observation rather than a method, since no detector was built from it, and building one is the first thing this project would do next.

6.5. The Usable Bandwidth Window

If the bandwidth sets the price of a boundary, it should be possible to make a boundary too expensive and also to make everything too cheap, and both happen in different ways. Table 4 sweeps σ on two concentric circles at $n = 600$ and noise 0.08.

The upper wall is a threshold rather than a slope, since between $\sigma = 0.0815$ and $\sigma = 0.0995$ the ARI falls from 0.980 to 0.711, while ρ, the share of edge weight crossing the true boundary, moves only from 0.20 per cent to 0.45 per cent. Sensitivity is the setup and not the explanation, since a quarter of a percentage point in a graph quantity does not by itself produce a collapse. The account of Section 4.4 attributes the collapse to the point at which the two regions are joined well enough that a function can ramp between them more cheaply than it can jump, so that the ramp takes the bottom place. That is consistent with the sweep, and the ramp and jump energies were not compared directly here, so it is offered as the reading rather than as a measurement.

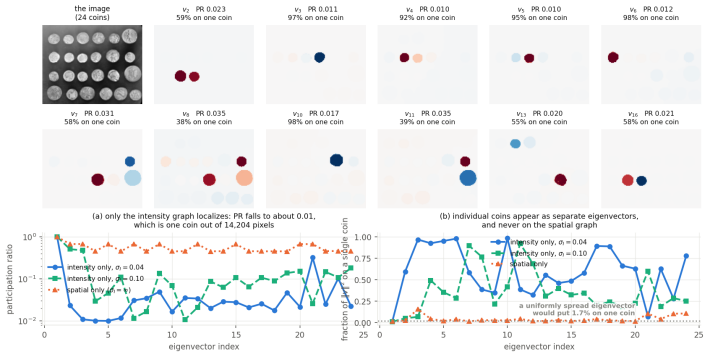


Figure 3. Eleven of the first sixteen eigenvectors of the intensity graph at $\sigma_I = 0.04$, drawn as images. Several of them sit on a single coin, which is where the object cuts should be found once the smooth modes are past. Panel (a) plots the participation ratio and panel (b) the energy on the best-matching coin, for the first 24 eigenvectors of three graphs.

Table 4. The two walls of the usable window, two circles, $n = 600$, noise 0.08, $r = 0.4$. Components are counted after treating weights below 10^{-12} as absent. In the upper block the graph has more pieces than clusters, so the last column is the best ARI obtainable from the components themselves, which upper-bounds anything the eigenvectors can return, and ρ is not meaningful. In the lower block the graph is a single connected object and the last column is the ARI returned by L_{rw}. The two blocks come from two sweeps on the same data.

σ	components	isolated points	ρ	best ARI available
0.0020	553	86%	—	0.000
0.0050	**378**	**43.3%**	—	**0.001**
0.0102	84	6.0%	—	0.064
0.0157	8	0.8%	—	0.960
0.0603	1	0%	7.4×10^{-4}	0.980
0.0815	1	0%	2.0×10^{-3}	0.980
0.0995	1	0%	4.5×10^{-3}	**0.711**
0.1101	1	0%	6.8×10^{-3}	**0.226**
0.1345	1	0%	1.5×10^{-2}	−0.001

The lower wall is the same mechanism running the other way, since at $\sigma = 0.005$ the graph has 378 components and 43.3 per cent of the points have no edges at all. Every isolated point is now an indicator of cost zero, so the bottom of the list fills with functions that describe single points rather than regions. The eigenvectors are exactly right about a graph that is no longer the data. This is the wall Prof. Murphy asked about when he suggested taking σ closer to zero, and the useful window between the two walls runs from 0.0157 to 0.0815 in this sweep, which is under one decade wide, and Figure 4 draws both of them.

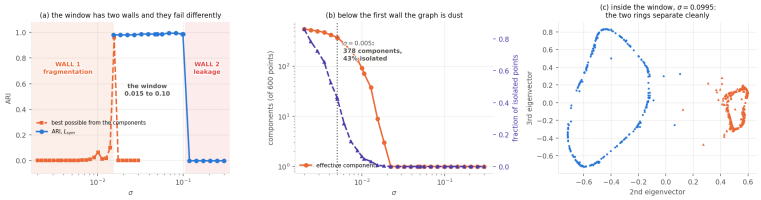


Figure 4. The two walls. Panel (a) plots ARI against σ with both walls shaded. Panel (b) is the lower wall, where at $\sigma = 0.005$ the graph has fallen into 378 components with 43.3 per cent of its points isolated, so the eigenvectors are correct about a graph that has become dust. Panel (c) is the embedding just below the upper wall, drawn from L_{sym}, which still separates the rings at a bandwidth where L_{rw} has already fallen to 0.711.

The three Laplacians also fail in order and not together. At $\sigma = 0.0995$ the unnormalized Laplacian scores 0.175, the random-walk normalization 0.711 and the symmetric normalization 0.987, so the symmetric

normalization stands 0.276 above the random-walk normalization at this bandwidth and the ordering von Luxburg [30] recommends is not the one measured.

6.5.1. Hypothesis Verdicts

Testing H1. The argmax of the eigengap falls at $\sigma = 0.41$, where the gap is 3.30×10^{-3} and the ARI is -0.002, against gaps of 1.32×10^{-3} and 1.46×10^{-3} at the two bandwidths that reach the highest ARI of 0.993 on a finer σ grid than Table 4 shows. **Verdict: refuted as a selector, confirmed as a diagnostic.**

The reason follows from Section 4 and not from a defect in the heuristic, since a merged graph is a perfectly good graph with its own clean bipartition and therefore has a large gap. The gap reports stability rather than correctness. Used as a selector its argmax falls at $\sigma = 0.41$, where the ARI is -0.002, so it is a diagnostic and not a selector.

Testing H2. The collapse is a threshold rather than a slope, as the table above shows. However, at $n = 600$, 1200 and 2400 the width of the transition broadens instead of narrows, from 0.035 to 0.070 to 0.105 decades. **Verdict: threshold clause confirmed, sharpening clause refuted.**

No mechanism is offered for the broadening, and the sharpening clause is recorded as a failure.

6.6. Bottleneck Recovery and Spectral Ordering

Two regions of identical texture joined by a bridge of tunable width are the case where only connectivity can help. Table 5 reports how often four grouping policies recover the bottleneck, over eight seeds at each width.

Table 5. Success rate at recovering a narrow bottleneck, 8 seeds per width. The breadth-first policy is the one Shi and Malik [23] recommend, which is to split every eligible segment instead of chasing the globally cheapest cut.

Bridge width (px)	0	2	4	8	14	22	Overall
simultaneous k-way	1.00	0.88	0.12	0.00	0.00	0.00	0.333
recursive, cheapest-first	1.00	1.00	1.00	0.62	0.12	0.12	0.646
recursive, largest-first	0.88	0.88	0.00	0.38	0.00	0.12	0.375
recursive, breadth-first	**1.00**	**1.00**	**1.00**	**0.62**	**1.00**	**1.00**	**0.938**

Testing H4. Breadth-first recursion recovers the bottleneck in 93.8 per cent of trials against 33.3 per cent for simultaneous k-way clustering, over eight seeds at each of six bridge widths, and Table 6 gives the reason in the ordering. **Verdict: confirmed.**

The success rates alone do not say why, so the spectrum was computed as well.

Table 6. The bottleneck, read out of the spectrum. The separating eigenvector is the first one whose mean value has opposite signs on the two same-texture regions and which places more than 30 per cent of its energy on their union. Success is the same test as Table 5, a pair-ARI above 0.5 on those two regions. At $k = 5$ the solver receives v_1 through v_5. The graph construction differs from Table 5, so the success columns of the two tables are not directly comparable.

Bridge width	Bandwidth	Ncut of the bridge cut	λ_2	First separating eigenvector	Pair-ARI at $k = 5$
4 px	median	2.42×10^{-3}	2.04×10^{-12}	v_5	0.997
4 px	connectivity-checked	2.42×10^{-3}	3.04×10^{-4}	v_5	0.992
14 px	median	6.44×10^{-3}	2.20×10^{-12}	v_7	**0.000**
14 px	connectivity-checked	6.44×10^{-3}	2.94×10^{-4}	v_5	0.978
22 px	median	9.32×10^{-3}	5.80×10^{-13}	v_7	**0.000**
22 px	connectivity-checked	9.32×10^{-3}	2.84×10^{-4}	v_6	**0.000**

The bridge cut is cheap and is still not at the bottom of the ordering. Its cost runs from 2.42×10^{-3} to 9.32×10^{-3} as the bridge widens, while on the connectivity-checked rows λ_2 stays between 2.84×10^{-4} and

3.04×10^{-4}, which is between 8.0 and 32.8 times cheaper and belongs to a ramp and not to any partition.

The last two columns hold the index of the separating eigenvector and whether $k = 5$ recovered the bottleneck, and they require careful reading. Across all six rows simultaneous k-way clustering at $k = 5$ recovers the bottleneck exactly when the separating eigenvector is v_5 or lower and fails whenever it is v_6 or higher. Half of that is definitional, because $k = 5$ hands the solver v_1 through v_5 and an eigenvector at index six is not in the embedding at all. The half that is not definitional is the first one, which is that the bandwidth moves the index. At 14 pixels the median rule puts the separating function at v_7 and the connectivity-checked rule puts the same function, on the same image, at v_5. The index is therefore a property of the graph and not of the scene, and k is the parameter that decides how much of the ordering the solver is allowed to see. The same account would explain what recursion does, since a smaller subgraph has no long lattice for a ramp to spread across, but that step was not measured and is offered as the reading.

6.7. Feature Ablation on Textured Regions

The first benchmark this project built used flat colored regions and failed in two ways that share one cause. It could not discriminate between feature sets, since every variant scored between 0.565 and 0.568, and at the median bandwidth its regions were disconnected to machine precision. Flat regions are not images, and the replacement composites real photographic texture, which is grass, gravel, brick and a crop of a color photograph, into regions whose labels are exact by construction, with two regions carrying the same texture and tint and joined by a bridge. The result is semi-synthetic and is not a substitute for human-segmented photographs.

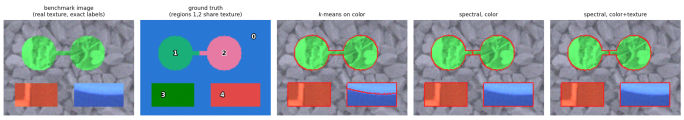


Figure 5. The benchmark and three outputs. The first two panels are the image and the ground truth, in which two regions share texture and tint so that only connectivity separates them. The remaining three outline the partitions returned by k-means on color, spectral clustering on color, and spectral clustering on color and texture.

Table 7. Feature ablation over 8 image seeds, mean and 95 per cent Student-t interval. σ_I is set by the median rule in every row. All five methods were run on the same eight images, so the rows are paired. The intervals are symmetric by construction and are not clipped at the upper bound of ARI, so the top of the spectral interval lies above the maximum attainable value of 1.

Method	Features	ARI	Boundary-F
k-means	color	0.664 ± 0.157	0.575 ± 0.201
k-means	color + position	0.521 ± 0.002	0.925 ± 0.031
spectral L_{rw}	texture only	0.064 ± 0.012	0.189 ± 0.026
spectral L_{rw}	color	$\mathbf{0.881 \pm 0.184}$	$\mathbf{0.992 \pm 0.013}$
spectral L_{rw}	color + texture	$\mathbf{0.881 \pm 0.184}$	$\mathbf{0.992 \pm 0.012}$

The graph scores higher here, at 0.881 ± 0.184 in ARI against 0.664 ± 0.157. The five methods were run on the same eight images, so the comparison is paired and is tested as such rather than by reading the intervals against each other. The median paired difference is 0.412 in favor of spectral clustering, and a Wilcoxon signed-rank test over the eight pairs returns $p = 0.20$, so the ARI column favors the graph without resolving the comparison at this sample size. The boundary-F column provides substantially stronger evidence in favor of spectral clustering, at 0.992 ± 0.013 against 0.575 ± 0.201, a difference of 0.417 between intervals that do not overlap, and it is the comparison this study relies on. Figure 5 shows the benchmark that produced both. Three of the five rows carry information beyond that comparison. Texture alone scores 0.064, because the

grass and gravel textures used have similar energy statistics, so a texture-energy channel does not separate regions that color already separates. Adding texture to color changes nothing to three decimal places, which is a null result and is reported as one. The two scores also disagree with each other inside the k-means rows, since adding position to color costs 0.143 in ARI and gains 0.350 in boundary-F, so a conclusion drawn from either score alone would be a conclusion about the score.

6.8. Computational Cost and Resolution Scaling

Runtime is dominated by the eigensolve, and every timing reported here is the eigensolve alone. ARPACK in shift-invert mode does not iterate toward the bottom of the spectrum. It factorizes a shifted matrix once and then iterates cheaply, so the exponent that governs the runtime belongs to sparse Gaussian elimination and not to spectral clustering.

H3 was testing that exponent. Nested dissection [16] on a two-dimensional lattice eliminates around a separator of side $\sqrt{n}$, which fills into a dense block costing $n^{1.5}$ [9], while in three dimensions the separator is a plane of side $n^{2/3}$ giving n^2.

Testing H3. The measured exponents are 2.00 ± 0.33 in three dimensions and 1.05 ± 0.13 on a four-neighbor two-dimensional lattice, against 2 and 1.5 predicted. Widening the radius on the same lattice walks the two-dimensional exponent to 1.45 at $r = 1.5$ and to 2.00 at $r = 8$, which is a second refutation of the hypothesis as stated, since the exponent turns out to be a property of the sparsity pattern the radius produces and not of the dimension. **Verdict: three-dimensional half held, two-dimensional half failed.**

The two-dimensional half failed because the library orders with COLAMD [7] instead of nested dissection, and because at these sizes memory traffic and not arithmetic bounds the run. The radius sweep above is the sharper statement of the same point. A four-neighbor lattice, a radius of 1.5 and a radius of 8 are the same problem at the same sizes, and they give exponents of 1.05, 1.45 and 2.00, so a reader quoting a complexity for this method is quoting a property of the graph they happened to build.

Two approximations were tested and neither improves on direct factorization, since a converged matrix-free solver [13] runs 1.4 to 1.9 times slower than factorizing and uniform-landmark Nyström [8] scores 0.017 to 0.300 ARI at 17 times the cost.

Coarsening improves both quantities at once. Clustering a half-resolution image and upsampling the labels drops the eigensolve from 7.27 to 0.99 seconds, a factor of 7.3, and raises ARI from 0.345 ± 0.004 to 0.960 ± 0.003. Accuracy and speed are not traded against each other over that range, and this study cannot say why.

6.8.1. Coarsening Without a Supporting Mechanism

The account of Section 4.4 suggests an explanation. Within-region texture variation creates cheap gradual functions inside a region, those compete for the bottom of the ordering against the region's own boundary, and coarsening deletes the variation. However, that explanation was tested twice and failed twice.

H5 was the first test. If shortening the lattice reduces the number of ramps that fit below a given energy, coarsening should move object-shaped eigenvectors down the ordering.

Testing H5. Repeating the coins experiment at four scales, the first eigenvector carrying more than half its energy on one coin appears at index 4, 4, 10 and 16 as the image shrinks from 14,204 to 3,551 pixels, which is the wrong direction. The second and more direct test, below, also came out against the explanation. **Verdict: refuted twice.**

The second test measures the competition directly, and it was run twice because the first version of it compared the wrong things. Inside one fixed region, λ_2 of the induced subgraph is the price of the cheapest ramp available in that region, and the Ncut of the same region against the rest is the price of its own boundary. Holding the bandwidth at its full-resolution value and scoring the same region at every scale, the ratio of the second to the first runs 0.64, 0.70, 1.95 and 1.30 at 100, 70, 50 and 35 per cent of full resolution. The prediction was that this ratio should fall. It rises instead, and it starts below one at 0.64, which indicates

that at full resolution the cheapest ramp inside the region already costs more than the region's own boundary. There was no internal competition there to remove, so the explanation fails at its first step rather than at its conclusion.

Both tests came out negative, and the coarsening result is real and repeated over five seeds, so the mechanism this study argues for does not account for it. It is the largest open question this project has, and no third explanation is offered.

6.8.2. Reevaluation of the Sparsification Claim

Shi and Malik [23] write that "in the cases of the large neighborhood size, we can randomly select 10% of the connections and still obtain a good segmentation." An earlier version of this project's own analysis put the survivable fraction at approximately 30 per cent. That reading was wrong, and it was wrong because two conditions in that sentence had been dropped. The procedure is random sampling inside each neighborhood and not removal of the weakest edges, and the neighborhoods have to be large. Re-run as written at mean degree 287, random sampling at 90 per cent removal agrees with the full-graph partition at ARI $= 0.9996$ over three seeds, with an eigenspace rotation of 1.3 degrees. Removing the weakest edges instead, which is the version the interim report tested, has already fallen to 0.65 agreement at 70 per cent and 0.48 at 80 per cent.

6.9. Unweighted Graphs from Occupancy Grids

Everything above is about what a weight function does to an ordering, so the control condition is a graph that has no weight function. A robot's occupancy grid is binary: a free cell becomes a vertex, a wall is absent, and adjacent free cells are joined with unit weight, so there is no bandwidth and nothing to tune. Figure 6 shows the map this was run on.

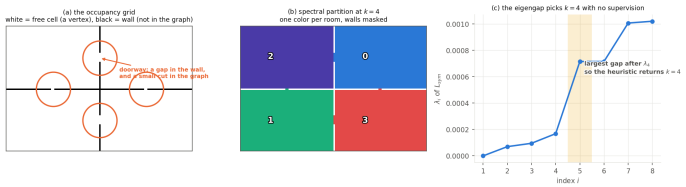


Figure 6. The four-room map. In the first panel a white cell is free and becomes one vertex, a black cell is a wall and is absent from the graph, and the doorways are circled. The second panel is the partition at $k = 4$ with walls masked, and the third plots the bottom eigenvalues, whose largest gap falls after λ_4. The graph has 9,417 vertices, unit weights and no parameters.

On a two-room map, the partition returned by the Fiedler vector scores between 0.960 and 0.999 ARI against the true rooms across door widths, and λ_2 grows monotonically by a factor of 4.7 as the doorway widens, which is the behavior the graph Cheeger inequality [1] leads one to expect. On the four-room map with $n = 9{,}417$ the bottom eigenvalues are 0, 6.98, 9.52, 16.8, 71.5 and 71.5, in units of 10^{-5}. The gap after the fourth eigenvalue is 5.47×10^{-4} against 7.28×10^{-5} for the largest gap below it, a factor of 7.5, so the eigengap selects $k = 4$ without supervision and the resulting partition scores ARI $= 0.995$ against the rooms.

The eigengap failed in Section 6.5 and works here, on the same heuristic and the same code, although the two settings differ in more than the weight function and this is therefore a contrast rather than a controlled comparison. Two toy maps are not a benchmark, so this is where the method would be tried first rather than where it is recommended.

7. Limitations and Remaining Questions

Halving the resolution runs 7.3 times faster and raises ARI from 0.345 to 0.960. That is the most useful practical finding here, and the mechanism argued for in Section 4 does not explain it. **H5** was written to test the obvious explanation and it failed, and a second and more direct test of the same explanation failed as well.

The second failure is the more damaging of the two, since at full resolution the cheapest ramp inside a region already costs more than that region's own boundary, at a ratio of 0.64, so the competition the explanation depends on is not there to be removed. A correct account of coarsening is the piece of theory this project is missing.

The next limitation is the data. The benchmark uses real photographic texture with labels that are exact by construction, and it is still a composite assembled by the author whose region shapes are geometric and whose boundaries are clean. The Berkeley segmentation data set [17] and its successor BSDS500 [2] contain photographs segmented by people, and neither could be obtained in the environment this work ran in, so every claim here is a claim about the graphs described and not about natural images in general.

On the counting rule of Section 4.7, only **H4** came through intact. **H1** survives as a diagnostic and fails as a selector. **H2** and **H3** each kept one clause and lost the other, and **H5** failed twice, which is the failure that cost this study an explanation for its most useful result.

Two conclusions of this project were reversed rather than refined. The claim that only approximately 30 per cent of graph edges could be dropped was wrong, and so was the claim that the normalized cut outranks small objects through its denominator, which a measurement with the opposite sign replaced. The second reversal is the more serious one, since that claim had reached an abstract before anyone computed the objective on the partitions it was about.

The audit also identified two coding defects, both recorded because they changed numbers. Connected components were once counted without a magnitude threshold, so a graph whose weakest edges were of order 10^{-78} was reported as connected, and confidence intervals were once computed with a normal quantile where the Student-t quantile is correct.

Two things were not varied. The cluster-selection half of Zelnik-Manor and Perona [31] was not tested, although it is the direct competitor to the eigengap result reported here, and the deep-feature descendants were discussed rather than run, because the pretrained weights were not reachable from this environment. The strongest available substitute is the five dimensional feature space of Section 6.3.4.

8. Conclusion

Section 1.1 began with a robot that could not separate two touching artifacts of the same color, and with the realization that no threshold on color ever could. Reframing that failure as a question about connectivity leads to spectral clustering, and asking one question of the method produced one answer.

The answer is that the eigenvectors handed to k-means are the cheapest functions on the graph rather than the cheapest cuts, and the two orderings differ because Dirichlet energy is quadratic. On the coins image that gap is a factor of 6.0 in the objective, and on a different scene with a five dimensional feature space it is between 3.2 and 3.9. At a bandwidth that makes object boundaries cheap enough, five of the first ten eigenvectors each carry between 92 and 98 per cent of their energy on one single coin, so the objects were outranked rather than absent.

The bandwidth is what sets the price of a boundary, and it therefore decides which of the two families of function reaches the bottom of the list first. The position in the ordering at which a bottleneck becomes visible follows from that one sentence and was measured across six settings, and both walls of the usable window are read the same way, although the two energies behind that reading were never compared directly. The coarsening result is not read that way at all.

The practical corollary is that the graph earns its runtime only when the structure that matters is defined by connectivity rather than by appearance, which is the case that defeated the robot in Section 1.1. Where the structure is convex the graph adds nothing, since k-means matched it at $\mathrm{ARI} = 1.000$ and at approximately one hundredth of the runtime. It is not usable yet inside a perception loop, since the eigensolve alone takes 7.27 seconds at $n = 14{,}204$ against the 33 milliseconds a thirty-frame-per-second budget allows. For the robot this work started from the answer is no, by a factor of approximately 220. What the graph did buy was an explanation of why the cheap method could never have worked.

Future Work

Coarsening needs a correct mechanism, and since the two explanations tried here are both dead, the next attempt should start from what changes about the *boundary* instead of what changes inside a region. The localization result should be a detector, since thresholding the participation ratio would harvest eigenvectors that already carry 98 per cent of their energy on one object. Every claim here should also be repeated on human-segmented photographs.

Acknowledgements

I thank Prof. James M. Murphy for the question that produced Section 6.3, for predicting the localization result before it was measured, and for telling me that a long paper full of experiments is not understanding.

References

[1] N. Alon and V. D. Milman. λ_1, isoperimetric inequalities for graphs, and superconcentrators. *Journal of Combinatorial Theory, Series B*, 38(1):73–88, 1985.

[2] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik. Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, 2011.

[3] M. Belkin and P. Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. In *Advances in Neural Information Processing Systems (NIPS)*, volume 14, pages 585–591, 2001.

[4] R. R. Coifman and S. Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006.

[5] M. Cucuringu and M. W. Mahoney. Localization on low-order eigenvectors of data matrices. *arXiv preprint arXiv:1109.1355*, 2011.

[6] C. Davis and W. M. Kahan. The rotation of eigenvectors by a perturbation. III. *SIAM Journal on Numerical Analysis*, 7(1):1–46, 1970.

[7] T. A. Davis, J. R. Gilbert, S. I. Larimore, and E. G. Ng. Algorithm 836: Colamd, a column approximate minimum degree ordering algorithm. *ACM Transactions on Mathematical Software*, 30(3): 377–380, 2004.

[8] C. Fowlkes, S. Belongie, F. Chung, and J. Malik. Spectral grouping using the nyström method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(2):214–225, 2004.

[9] A. George. Nested dissection of a regular finite element mesh. *SIAM Journal on Numerical Analysis*, 10(2):345–363, 1973.

[10] S. Guattery and G. L. Miller. On the quality of spectral separators. *SIAM Journal on Matrix Analysis and Applications*, 19(3): 701–719, 1998.

[11] W. He, S. Zhang, C.-G. Li, X. Qi, R. Xiao, and J. Guo. Neural normalized cut: A differential and generalizable approach for spectral clustering. *Pattern Recognition*, 2025. arXiv:2503.09260.

[12] L. Hubert and P. Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985.

[13] A. V. Knyazev. Toward the optimal preconditioned eigensolver: Locally optimal block preconditioned conjugate gradient method. *SIAM Journal on Scientific Computing*, 23(2):517–541, 2001.

[14] F. Krzakala, C. Moore, E. Mossel, J. Neeman, A. Sly, L. Zdeborová, and P. Zhang. Spectral redemption in clustering sparse networks. *Proceedings of the National Academy of Sciences*, 110(52):20935–20940, 2013.

[15] R. B. Lehoucq, D. C. Sorensen, and C. Yang. *ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*. SIAM, 1998.

[16] R. J. Lipton and R. E. Tarjan. A separator theorem for planar graphs. *SIAM Journal on Applied Mathematics*, 36(2):177–189, 1979.

[17] D. Martin, C. Fowlkes, D. Tal, and J. Malik. A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In *Proceedings of the 8th IEEE International Conference on Computer Vision (ICCV)*, pages 416–423, 2001.

[18] L. Melas-Kyriazi, C. Rupprecht, I. Laina, and A. Vedaldi. Deep spectral methods: A surprisingly strong baseline for unsupervised semantic segmentation and localization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8364–8375, 2022.

[19] R. R. Nadakuditi and M. E. J. Newman. Graph spectra and the detectability of community structure in networks. *Physical Review Letters*, 108:188701, 2012.

[20] A. Y. Ng, M. I. Jordan, and Y. Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems (NIPS)*, volume 14, pages 849–856, 2001.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[22] A. Saade, F. Krzakala, and L. Zdeborová. Spectral clustering of graphs with the Bethe Hessian. In *Advances in Neural Information Processing Systems (NIPS)*, volume 27, 2014.

[23] J. Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.

[24] G. Strang. *Linear Algebra and Learning from Data*. Wellesley-Cambridge Press, 2019.

[25] M. Tang, A. Djelouah, F. Perazzi, Y. Boykov, and C. Schroers. Normalized cut loss for weakly-supervised CNN segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1818–1827, 2018.

[26] N. Tremblay and A. Loukas. Approximating spectral clustering via sampling: A review. In *Sampling Techniques for Supervised or Unsupervised Tasks*, pages 129–183. Springer, 2019.

[27] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu. scikit-image: Image processing in python. *PeerJ*, 2:e453, 2014.

[28] N. X. Vinh, J. Epps, and J. Bailey. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, 11: 2837–2854, 2010.

[29] P. Virtanen et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

[30] U. von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007.

[31] L. Zelnik-Manor and P. Perona. Self-tuning spectral clustering. In *Advances in Neural Information Processing Systems (NIPS)*, volume 17, pages 1601–1608, 2004.